

AT32 BLDC Sensorless ADC 1-Shunt Demo

Introduction

The purpose of the application note is to demonstrate how to drive a three-phase permanent magnet synchronous motor (BLDC motor) with sensorless six-step square wave control algorithms. The contents include environment settings, program document architecture, macro definitions and motor control state machine.

ArteryTek has many motor development boards. In this document, AT-MOTOR-EVB is used for demonstration.

AT-MOTOR-EVB Applicable products:

Part number	AT32F402/405 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32L021 series
	AT32F45x series
	AT32M412/M416 series

Contents

1	Motor control library algorithm	5
2	Environment requirements	6
	2.1 Hardware requirements	6
	2.2 Software requirements	9
3	BLDC motor control library architecture.....	13
4	Motor control state machine.....	29
	4.1 Introduction.....	29
5	How to use demo project	33
6	Revision history.....	34

List of tables

Table 1. Motor specifications	6
Table 2. Flash memory size and ROM configuration	9
Table 3. Motor control library file list	15
Table 4. Drive mode definitions	16
Table 5. Control parameter definition.....	17
Table 6. Driver parameter definition.....	20
Table 7. Motor parameter definition	22
Table 8. Configuration of peripherals (AT32F413RCT7)	24
Table 9. Configuration of peripherals (AT32F421C8T7).....	24
Table 10. Peripheral configuration functions	25
Table 11. Motor control interrupt functions.....	26
Table 12. Motor control library emulation list	27
Table 13. State switch conditions	31
Table 14. Document revision history.....	34

List of figures

Figure 1. JK42BLS01-X056ED motor.....	6
Figure 2. AT-MOTOR-EVB evaluation board.....	7
Figure 3. AT32F413RCT7 ROM configuration (Keil).....	10
Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)	11
Figure 5. AT32F413RCT7 ROM configuration (IAR).....	12
Figure 6. Motor control program architecture	13
Figure 7. Structure of motor control files	13
Figure 8. Structure of motor control project.....	14
Figure 9. State machine flow chart.....	29

1 Motor control library algorithm

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)

Control modes:

- 120° square wave commutation

Three-phase PWM method:

- 120° conduction PWM control

Phase current sensing method:

- 1-shunt current sensing and reconstruction method

Initial rotor position detection mode:

- Three-phase voltage vector mode

Rotor position estimation in 120° square wave control mode:

- BEMF zero crossing point detection by ADC
- BEMF zero crossing point detection by comparator

Sensorless 120° square wave control mode:

- 120° square wave voltage control
- Torque control (120° square wave current control control)
- Speed control

Auto tuning functions:

- Motor winding parameters identification
- Current PI control parameters self-tuning

2 Environment requirements

2.1 Hardware requirements

A PMSM (BLDC) motor, AT-Link or third-party debugger, and motor control evaluation board (AT-MOTOR-EVB) are required. For more information on hardware configurations, refer to *UM0011_AT32_LV_Motor_Control_EVB_User_Manual*.

- PMSM (BLDC) motor: JK42BLS01-X056ED

Figure 1. JK42BLS01-X056ED motor

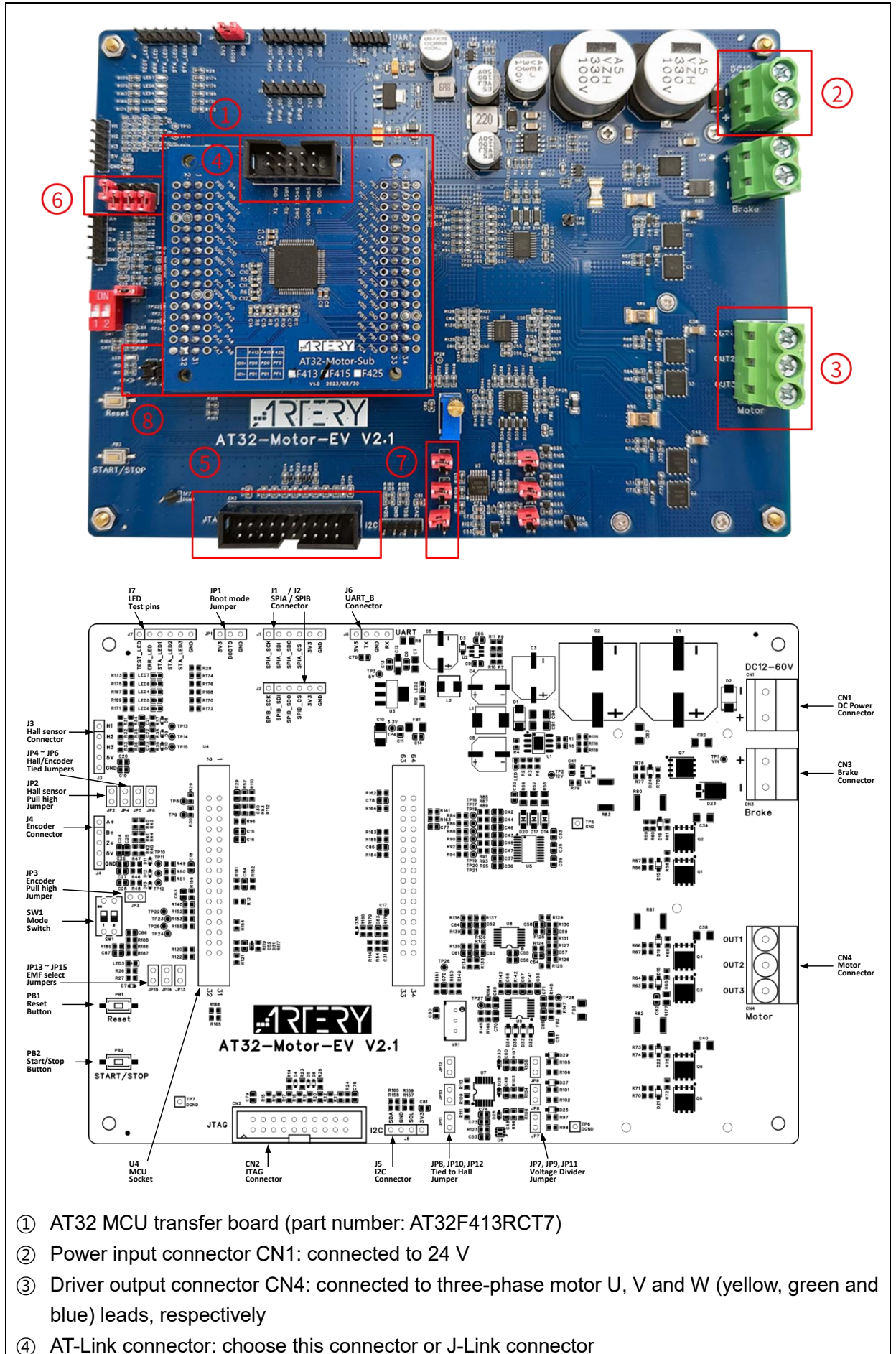


Table 1. Motor specifications

Item	JK42BLS01-X056ED
Motor poles	8
Rated voltage	DC 24V
Rated speed	4000 RPM
Rated torque	0.0716 N·m
No-load current	0.3 Amps
Output power	30 W
Line-to-line resistance	1.8Ω
No-load speed	6800 RPM
BEMF constant	4.36 V/Krpm
Torque constant	0.042 N·m/A
Encoder resolution	1000 P/R
Insulation grade	Class B
Winding type	Delta

- Debugger
- Motor control evaluation board: AT-MOTOR-EVB (embeds an AT32F413RCT7 MCU)

Figure 2. AT-MOTOR-EVB evaluation board



- ⑤ J-Link connector: choose this connector or AT-Link connector
- ⑥ Jumper: JP2, JP3, JP4, JP5, JP6 OPEN
- ⑦ Jumper: JP8, JP10, JP12 OPEN (BEMF zero crossing point detection by ADC)
JP8, JP10, JP12 CLOSED (BEMF zero crossing point detection by CMP)
- ⑧ Jumper settings: JP13, JP14, JP15 CLOSED

2.2 Software requirements

- 1) Open the “bldc_1shunt_sensorless” demo project.
- 2) Run “ArteryMotorMonitor.exe” (installation not required).
- 3) In Keil, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU (see Table 2). For example, for AT32F413RCT7 with a 256KB Flash memory, its IROM1 start address is at 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is at 0x803F800 with a size of 0x800 (see Figure 3). In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 4). In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 5).

Table 2. Flash memory size and ROM configuration

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: Keil compiler version 5 is recommended since AT32 BSP source code does not support V6.15 compiler.

Figure 3. AT32F413RCT7 ROM configuration (Keil)

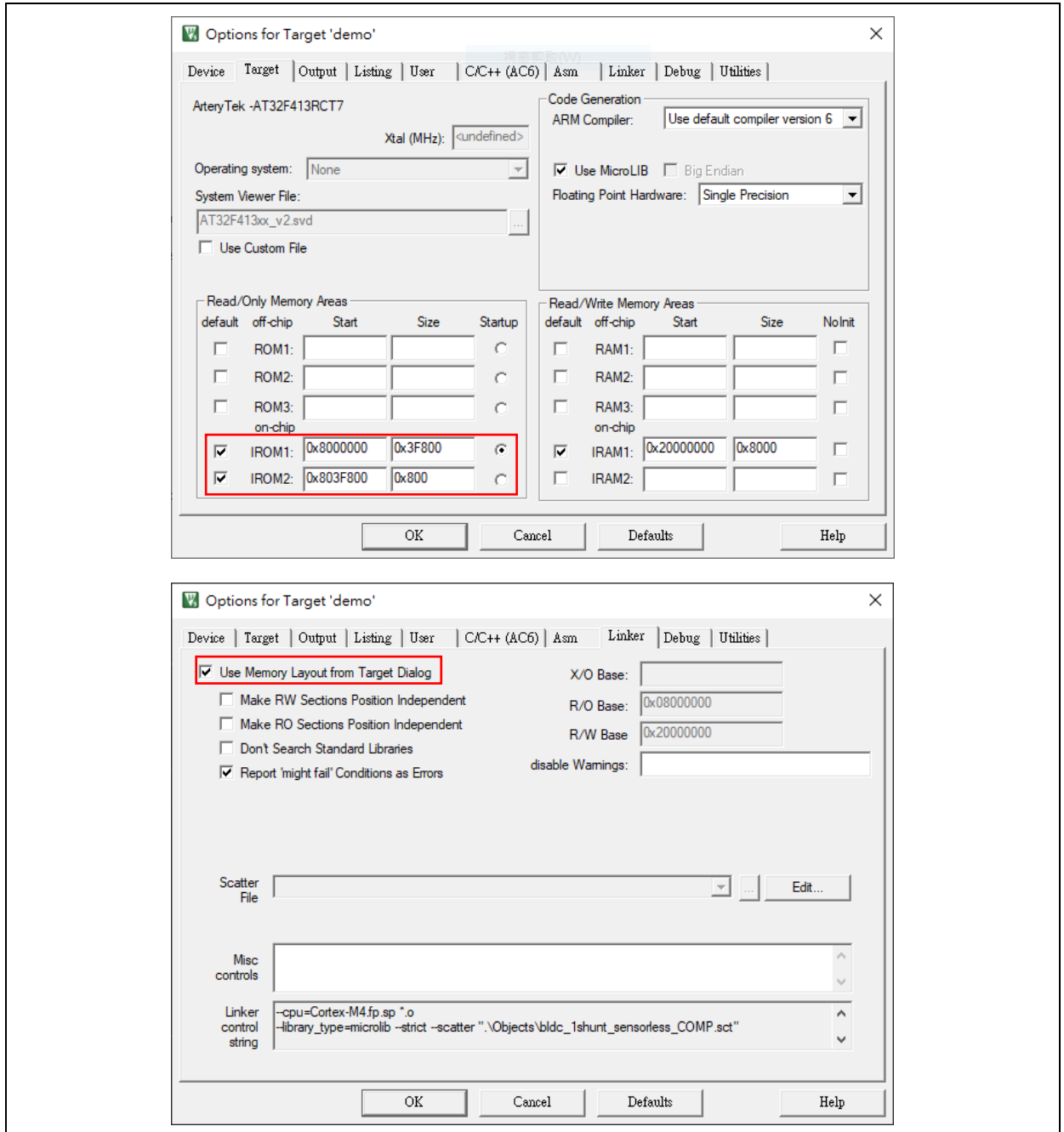


Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)

```

AT32F413xC_FLASH.ld X
25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34     FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
35     MC_DATA (r)      : ORIGIN = 0x0803F800, LENGTH = 0x800
36     RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42     /* The startup code goes first into FLASH */
43     .isr_vector :
44     {
45         . = ALIGN(4);
46         KEEP(*(.isr_vector)) /* Startup code */
47         . = ALIGN(4);
48     } >FLASH
49
50     .mc_data :
51     {
52         . = ALIGN(4);
53         . = ORIGIN(MC_DATA);
54         _MC_VectStoreAddr1 = .;
55         *(.MC_VectStoreAddr1);
56         . = ALIGN(4);
57         . = ORIGIN(MC_DATA) + 800;
58         _MC_VectStoreAddr2 = .;
59         *(.MC_VectStoreAddr2);
60         . = ALIGN(4);
61     } >MC_DATA
62
63     /* The program code and other data goes into FLASH */
64     .text :
65     {
66         . = ALIGN(4);
67         *(.text)           /* .text sections (code) */
68         *(.text*)          /* .text* sections (code) */
69         *(.glue_7)         /* glue arm to thumb code */
70         *(.glue_7t)        /* glue thumb to arm code */
71         *(.eh_frame)
72

```

Figure 5. AT32F413RCT7 ROM configuration (IAR)

```

1  /*###ICF### Section handled by ICF editor, don't touch! ****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F800 - 1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F800;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 800 - 1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = 0x0803F800 + 0x800 - 1;
13 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
14 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
15 /*-Sizes-*/
16 define symbol __ICFEDIT_size_cstack__ = 0x400;
17 define symbol __ICFEDIT_size_heap__ = 0x200;
18 /***** End of ICF editor section. ###ICF###*/
19
20 define memory mem with size = 4G;
21 define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
22 define region ROM1_region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
23 define region ROM2_region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
24 define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
25
26 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
27 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
28
29 initialize by copy { readwrite };
30 do not initialize { section .noinit };
31
32 place at address mem: __ICFEDIT_intvec_start__ { readonly section .intvec };
33
34
35 place in ROM_region { readonly };
36 place in ROM1_region { readonly section .MC_VectStoreAddr1 };
37 place in ROM2_region { readonly section .MC_VectStoreAddr2 };
38 place in RAM_region { readwrite,
39 block CSTACK, block HEAP };

```

3 BLDC motor control library architecture

Figure 6 presents the relationship between motor control functions, BSP, UI programs and hardware peripherals. In this motor control library architecture, the underlying hardware peripherals are controlled with BSP functions. The motor control functions and UI functions are built based on BSP and low-level functions. The user-defined programs are based on the motor control functions and UI functions. This organization makes it easier for users to call motor control functions to control MCU hardware peripherals and execute motor control programs. At the same time, it is also possible to obtain real-time motor control status or adjust motor control parameters by using UI control functions linked to the external PC UI software.

Figure 6. Motor control program architecture

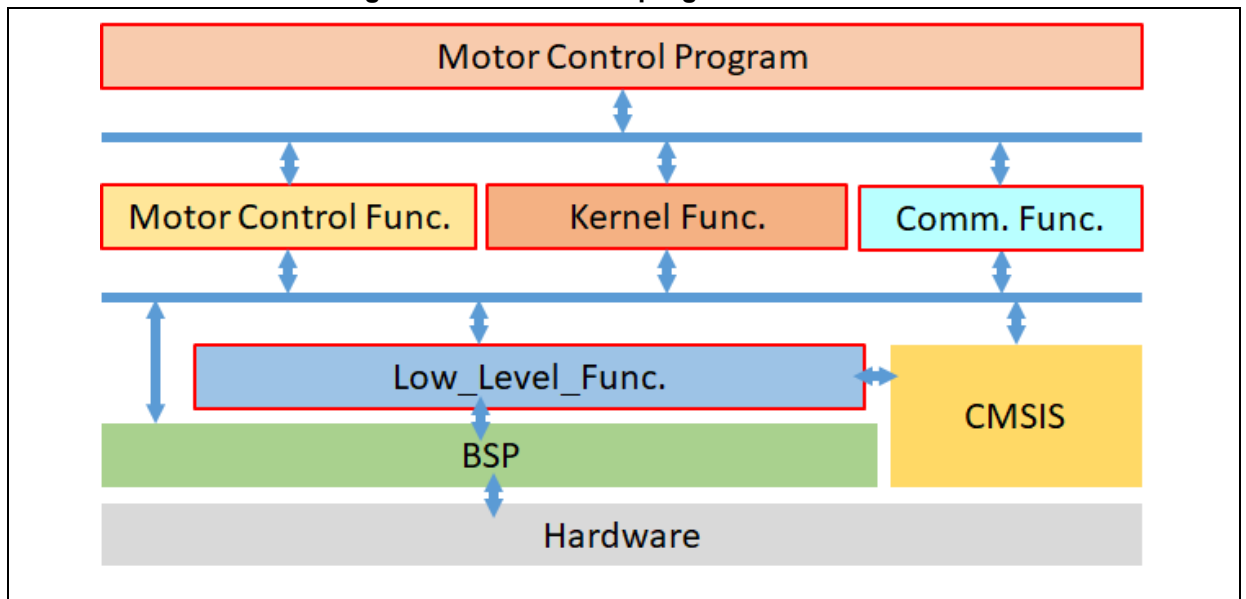


Figure 7 illustrates the overall structure of motor control files. The “motor_control_drive.h” header file is used to configure drive mode, control parameters, motor parameters and board parameters, while the “mc_hwio.h” header file is used to configure MCU peripheral parameters. All of these parameters are then integrated into the “mc_foc_globals.h (mc_bldc_globals.h)” and programmed with the initial variables through the “mc_foc_globals.c (mc_bldc_globals.c)”. The “mc_hwio.c” can be used to initialize MCU peripherals.

Figure 7. Structure of motor control files

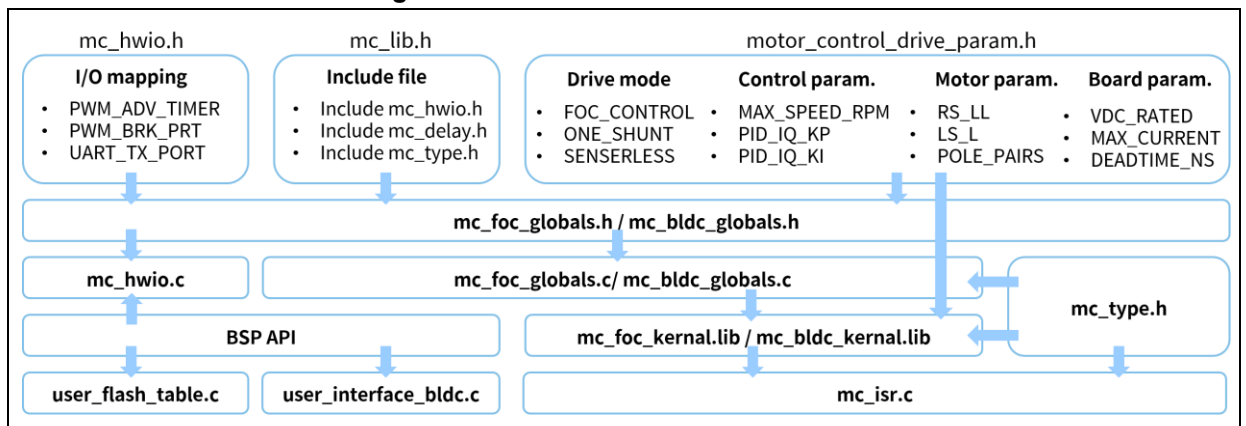


Figure 8 demonstrates the organization of files in the “bldc_1shunt_sensorless_ADC” demo project. The “user” is a user-defined folder containing the main programs, peripheral programs and parameter definition header files.

The “firmware” folder holds BSP files. The “mclib” folder is used to contain motor control library programs, including delay functions, communication functions, global variable setting functions and motor control library functions.

For details, refer to *AN0064 AT32_Motor_Control_Library_User_Guide* on Artery official website.

Figure 8. Structure of motor control project

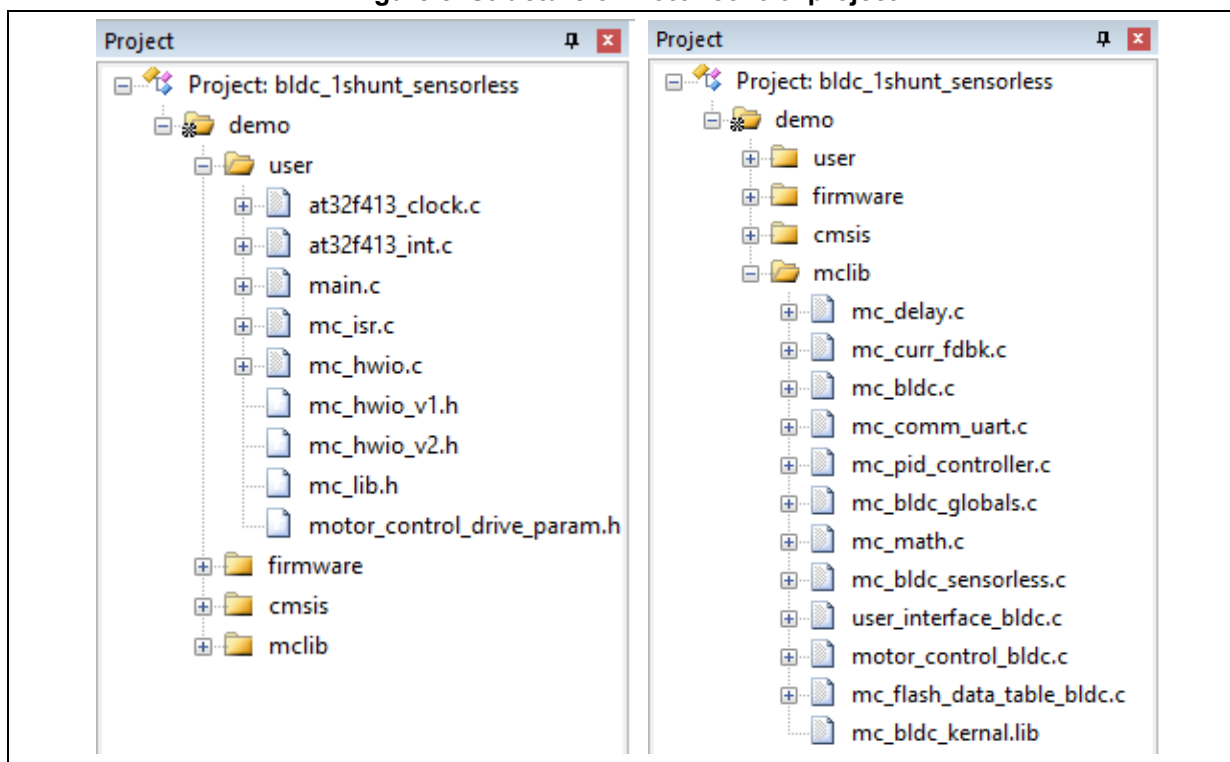


Table 3 gives a list of motor control library files.

Table 3. Motor control library file list

File name	Description
mc_lib.h	Header files
motor_control_drive_param.h	Set motor drive mode (current sampling mode, sensed mode, etc.), control parameters, drive parameters and motor parameters
mc_hwio.c	Hardware peripheral configuration
mc_hwio.h	Set hardware IO macro definition
mc_isr.c	Motor control interrupt functions
mc_type.h	Global variables type and definition, enumeration definition
mc_delay.c	Delay functions
mc_delay.h	Declaration of delay functions
mc_comm_uart.c	Communication peripheral configuration
mc_comm_uart.h	Declaration and configuration of UART functions
mc_pid_control.c	PID controller functions
mc_pid_control.h	Declaration of PID controller functions
mc_curr_fdbk.c	Current sensing functions
mc_curr_fdbk.h	Declaration of current sensing functions
mc_math.c	Filter functions
mc_math.h	Declaration of filter functions
mc_bldc_sensorless.c	Sensorless functions
mc_bldc_sensorless.h	Declaration of sensorless functions
motor_control_bldc.c	Motor control functions
motor_control_bldc.h	Declaration of motor control functions
mc_bldc.c	Six-step square wave functions
mc_bldc.h	Declaration of six-step square wave functions
mc_bldc_globals.c	Global variables definition and default value, global function definition
mc_bldc_globals.h	Global variables, global function declaration, macro definitions
user_interface_bldc.c	Communication interface functions
user_interface_bldc.h	Declaration of communication interface functions
mc_flash_data_table_bldc.c	Write Flash data table
mc_flash_data_table_bldc.h	Write Flash data table configuration

1) **motor_control_drive_param.h** file

- This file is divided into four parts and used to configure drive mode, control parameters, motor parameters and board parameters.
- Table 4 is a list of drive mode macro definitions. The user can use them to define the desired mode. This demo project uses the following features: PWM output complementary, PWN lower-side output inverted, sensorless mode, zero-crossing detection with ADC, low-speed voltage control mode, phase advance, startup at constant voltage, align and go startup sensorless control mode, winding parameters auto identification, and Artery motor UI tool.

Table 4. Drive mode definitions

Macro definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)
AT_MOTOR_EVB_V1	Motor control evaluation board V1.x
AT_MOTOR_EVB_V2	Motor control evaluation board V2.x
COMPLEMENT	Enable complementarity between low-side MOS transistor switch and upper-side PWM (enabled by default; comment to disable)
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output (enabled by default for AT32-Motor-EV; disabled by default for AT32M412-LV-Motor-EV)
EMF_COMPENSATE	BEMF voltage compensate
SENSORLESS	Sensorless control mode (general-purpose)
BLDC_SENSORLESS_ADC	BEMF detection with ADC (enabled this feature or BLDC_SENSORLESS_COMP)
BLDC_SENSORLESS_COMP	BEMF detection with comparator (enabled this feature or BLDC_SENSORLESS_ADC)
WITHOUT_CURRENT_CTRL	Voltage control without current loop (disabled by default; comment to enable)
LOW_SPEED_VOLT_CTRL	Low-speed voltage control mode (enabled by default; comment to disable)
PHASE_ADVANCE	Phase advance (enabled by default; uncomment to disable)
LARGE_COGGING	Large cogging torque (in sensorless six-step square wave control mode)
CONST_CURRENT_START	Startup at constant current (in six-step square wave sensorless control mode)
CONST_VOLTAGE_START	Startup at constant voltage (in six-step square wave sensorless control mode)
INIT_ANGLE_STARTUP	Startup at initial angle detection in sensorless control mode (general-purpose)
ALIGN_AND_GO_STARTUP	Align and go startup in sensorless control mode (general-purpose)
OPENLOOP_STARTUP	Open loop startup in sensorless control mode (general-purpose)
MOTOR_PARAM_IDENTIFY	Motor winding parameters identification
USE_MOTOR_MONITOR	Using Artery's Motor Control UI Features (enabled by default; comment to disable)

For BLDC_SENSORLESS_COMP only	
EMF_CONTINUOUS_SAMPLE	Continuous sampling mode for BEMF zero-crossing detection (disabled by default; comment to enable)
SPECIFIC_EACH_EMF_PIN	Shield signals from other comparators (enabled by default; comment to disable)
For BLDC_SENSORLESS_ADC only	
EMF_PULL_UP	ARTERY's proprietary pull-up circuit dedicated to enhance BEMF sampling and analysis with ADC in low-speed mode (Patent) (disabled by default; comment to enable)

- Table 5 presents the motor control parameters that are used in this demo. The user can set the desired parameters according to application needs, such as Is current PI controller, speed PI controller, etc.

Table 5. Control parameter definition

Macro definition	Description
PWM_FREQ	PWM output frequency (unit: Hz)
MOTOR_CONTROL_MODE	Motor control mode (speed control, torque control, etc.; see motor_control_mode in the <i>type.h</i> for details)
CTRL_SOURCE	Control source settings (external source control/software control)
UI_UART_BAUDRATE	Baud rate of UI communication serial port (preset value: 1.5M)
TUNE_TARGET_CURRENT	Target current value for tuning current loop PI control parameters (unit: ampere)
TUNE_CURRENT_TOTAL_PERIOD	Total period for tuning current pulse for current loop PI control parameters (unit: ms)
TUNE_CURRENT_STEP_PERIOD	Step period for tuning current pulse for current loop PI control parameters (unit: ms)
I_SAMP_MIN_TIME	Minimum time for sampling ADC values (unit: nsec)
I_SAMP_DELAY	Drive signal delay time plus current sensing signal delay time (unit: nsec)
SENSE_HALL_TIMES	Set phase-change times from open loop back to closed loop (unit: times)
REBOOT_PERIOD_MS	Set a restart time when a startup failed (unit: ms)
START_CURRENT	Current value for constant current startup (unit: ampere)
START_VOLTAGE	Voltage value for constant voltage startup (unit: V)
SPEED_FILTER_TIMES	Speed low-pass filter timers (unit: times)
SPD_LP_BANDWIDTH	Speed low-pass filter band width (unit: Hz)
PWM_IN_FILTER_TIMES	PWM pulse width low-pass filter times (unit: times)

Macro definition	Description
SPD_CMD_FILTER_TIMES	Speed command low-pass filter times (unit: times)
MIN_SPEED_RPM	Minimum motor speed (unit: rpm)
MAX_SPEED_RPM	Maximum motor speed (unit: rpm)
MAX_CCW_SPEED_RPM	Maximum motor reverse speed (unit: rpm)
ACC_SPD_SLOPE	Slope of acceleration (unit: rpm/ms, when speed control loop =1kHz)
DEC_SPD_SLOPE	Slope of deceleration (unit: rpm/ms, when speed control loop =1kHz)
MIN_SENSE_SPEED	Minimum speed for speed loop control (unit: rpm)
PWM_DUTY_TMR_DIV	Clock divider for detecting PWM pulse width input
PWM_IN_MAX_WIDTH	Maximum width of PWM input (unit: usec)
PWM_IN_NEUTRAL_WIDTH	Neutral width of PWM input (unit: usec)
PWM_IN_MIN_WIDTH	Minimum width of PWM input (unit: usec)
DEADZONE_PULSEWIDTH	Dead-zone width of PWM input (unit: usec)
TOLERANCE_PULSEWIDTH	Buffer width of PWM input (unit: usec)
SPD_CURVE_SWITCH_WIDTH	Switching interval width of PWM input (unit: usec)
SWITCH_SPD_RPM	Speed at the switching point of PWM input (unit: RPM)
BRAKE_DUTY	Brake PWM duty cycle (unit: PWM duty%)
DRAG_BRAKE_DUTY	Drag brake PWM duty cycle (unit: PWM duty%)
TONE_FREQ	Warning tone frequency (unit: Hz)
TONE_AMP	Warning tone volume
TONE_PERIOD	Duration of warning tone
SP_MAX_VOLT	Maximum voltage of external command source (unit: voltage)
SP_THRESHOLD	Threshold voltage of external command source (unit: voltage)
SP_RUN_VALUE	Minimum voltage to start running in external source mode (unit: voltage)
SP_STOP_VALUE	Maximum voltage to stop running in external source mode (unit: voltage)
OLC_INIT_SPD	Open loop initial speed (unit: rpm)
OLC_FINAL_SPD	Open loop final speed (unit: rpm)
OLC_TIMES	Set the times from initial speed to rise up to final speed (unit: times)
OLC_INIT_VOLT	Open loop initial voltage (unit: V)
OLC_VOLT_INC	Open loop incremental voltage (unit: V/time)
OLC_STARTUP_PERIOD	Set the period from open loop startup to closed loop (unit: ms) (for BLDC sensorless mode only)
LOCK_VOLT	Rotor alignment voltage before startup (Q15 mode)

Macro definition	Description
LOCK_PERIOD	Rotor alignment time before startup (unit: ms)
PID_IS_KP_DIV	Bus current control proportional gain divisor (Q16 mode)
PID_IS_KI_DIV	Bus current control integral gain divisor (Q16 mode)
PID_IS_KP_DEFAULT	Bus current control proportional gain (Q15 mode)
PID_IS_KI_DEFAULT	Bus current control integral gain (Q15 mode)
PID_SPD_KP_DIV	Speed control proportional gain divisor (Q16 mode)
PID_SPD_KI_DIV	Speed control integral gain divisor (Q16 mode)
PID_SPD_KP_DEFAULT	Speed control proportional gain (Q15 mode)
PID_SPD_KI_DEFAULT	Speed control integral gain (Q15 mode)
HYSTERESIS_LOW_SPEED	Minimum speed from current control to switch back to voltage control in low-speed voltage control mode (unit: rpm) (for LOW_SPEED_VOLT_CTRL only)
HYSTERESIS_HIGH_SPEED	Maximum speed from voltage control to switch back to current control in low-speed voltage control mode (unit: rpm) (for LOW_SPEED_VOLT_CTRL only)
PID_SPD_VOLT_KP_DEFAULT	Low-speed voltage control proportional gain (Q15 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KI_DEFAULT	Low-speed voltage control integral gain (Q15 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KP_GAIN_DIV	Low-speed voltage control proportional gain divisor (Q16 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KI_GAIN_DIV	Low-speed voltage control integral gain divisor (Q16 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
EMF_CHANGE_PERCENT_H	Duty value for BEMF zero-crossing detection to switch from low speed mode to high speed mode (unit: PWM counter timing base)
EMF_CHANGE_PERCENT_L	Duty value for BEMF zero-crossing detection to switch from high speed mode to low speed mode (unit: PWM counter timing base)
MIN_SAMPLE_INTERVAL_US	Minimum BEMF zero-crossing sampling interval (unit: usec)
EMF_SAMPLE_LEAD_PWM	BEMF zero-crossing sampling point DUTY value in low speed range (unit: PWM counter timing base) (for sensorless BLDC only)
EMF_HIGH_SPD_SAMPLE_CNT	BEMF zero-crossing sampling point DUTY value in high speed range (unit: PWM counter timing base) (for sensorless BLDC only)

Macro definition	Description
EMF_PHASE_ADV_SPD	Phase advance maximum speed (unit: rpm)
EMF_MIN_DELAY_US	Phase advance minimum delay (unit: usec)
EMF_AVOID_NOISE_INIT_PERIOD	Delay time for detecting BEMF zero-crossing point after initial period (unit: ms)
CURRENT_BANDWIDTH	Current PI controller bandwidth (dedicated for current PI controller parameters self-tuning)
For EMF_CONTINUOUS_SAMPLE only (detection with CMP)	
EMF_SAMPLE_INTERVAL	Sampling interval in continuous sampling mode (unit: PWM timer timing base) (for sensorless BLDC only)
SENSE_GPIN_DELAY	Read CMP GPIO delay time (unit: PWM timer timing base)
EMF_LOW_SPD_CONT_SAMPLE_END	Low-speed sampling end point in continuous sampling mode (unit: PWM timer timing base) (for sensorless BLDC only)
EMF_AVOID_NOISE_TIMES	Delay time for BEMF zero-crossing to avoid phase change noise (unit: PWM timer timing base)

- Table 6 presents the driver parameters that are used in this demo, including dead time, shunt resistance and current amplifier gain. Note that the corresponding parameters should be modified if a different motor drive board is to be used.

Table 6. Driver parameter definition

Macro definition	Description
VDC_RATED	DC-BUS voltage
BAT_LOW_VOLT	Lowest battery voltage (unit: voltage)
V_SENSE_GAIN	Voltage feedback ratio (refer to the corresponding motor control EVB board user manual)
MAX_CURRENT	Maximum peak phase current of motor (unit: ampere)
MIN_CURRENT	Minimum peak phase current of motor (unit: ampere)
CURRENT_SPAN_SHIFT	Number of shifts for current per-unit normalization
ADC_REFERENCE_VOLT	ADC reference voltage (unit: voltage)
ADC_DIGITAL_SCALE_12BITS	ADC resolution
SYSTEM_CORE_CLOCK	System frequency (unit: Hz)
TMR_CLK	Timer clock frequency (unit: Hz)
CHANGE_PHASE_TMR_DIV	Phase change clock frequency division (for six-step square wave sensorless mode)
DEADTIME_CLK_SFT_BITS	Shift bit number of frequency divider for dead time generator
DEADTIME_NS	Dead time (unit: ns)
R_SHUNT	Shunt resistance (unit: Ω)
OP_GAIN	Current amplifier gain (refer to the corresponding motor

Macro definition	Description
	control EVB board user manual)
CURR_OFFSET_VOLT	Zero current offset (refer to the corresponding motor control EVB board user manual) (unit: voltage)
EMF_SENSE_GAIN	BEMF feedback ratio (refer to the section 5.6 in the <i>UM0011_AT32_LV_Motor_Control_EVB_User_Manual</i>)
GATE_DELAY_TIME	Gate delay time (unit: ns)
EMF_SIG_RISING_TIME	Delay time for BEMF signal rising (unit: ns)
EMF_SIG_FALLING_TIME	Delay time for BEMF signal falling (unit: ns)
OVER_CURRENT_SW	Software overcurrent threshold point (unit: ampere)
DAC_VREF_SOURCE	Internal OP overcurrent protection DAC source selection (for AT32 MCUs with internal OPs, such as AT32M412)
OCP_CURRENT	Hardware overcurrent threshold point (unit: ampere)
BUS_CURR_CMP_OCP_VOLT	Internal OP overcurrent protection voltage (for AT32 MCUs with internal OPs, such as AT32M412) (unit: voltage)
TMR_BRK_FILTER_COUNT	Overcurrent protection break input filtering times (for AT32 MCUs with break input, such as AT32M412)
OVER_VOLT_THRESHOLD	Over-voltage threshold point (unit: voltage)
UNDER_VOLT_THRESHOLD	Under-voltage threshold point (unit: voltage)
V0_V	Parameter V0 of the approximate curve of NTC temperature vs. voltage (note [2])
T0_C	Parameter T0 of the approximate curve of NTC temperature vs. voltage (note [2])
dV_dT	Parameter dV/dT of the approximate curve of NTC temperature vs. voltage (note [2])
OVER_TEMP_THRESHOLD	Over-temperature threshold point (unit: Celsius degrees)
MC_ERROR_MASK	Error detection mask
For BEMF detection with internal CMP in sensorless BLDC mode (note [3])	
EMF_RISE_BLANK_TIME	Blanking window during PWM ON (unit: usec)
EMF_FALL_BLANK_TIME_HIGH_SPD	Blanking window duration when PWM is OFF during high-speed operation (Unit: μ s) (For sensorless BLDC internal comparator back-EMF detection method)
EMF_FALL_BLANK_TIME_LOW_SPD	Blanking window duration when PWM is OFF during low-speed operation (Unit: μ s) (For sensorless BLDC internal comparator back-EMF detection method)
EMF_BLANK_TIME_CHANGED_RPM	RPM threshold for switching between high-speed and low-speed blanking windows during PWM OFF state (Unit: rpm) (For sensorless BLDC internal comparator back-EMF detection method)
EMF_MIN_VALUE	Minimum BEMF for determining motor stop (ADC read value)

Macro definition	Description
For BEMF detection with ADC in sensorless BLDC mode	
EMF_LOW_SPD_OFFSET_RISING	BEMF zero-crossing point at rising edge in low-speed mode (unit: voltage digital conversion value)
EMF_LOW_SPD_OFFSET_FALLING	BEMF zero-crossing point at falling edge in low-speed mode (unit: voltage digital conversion value)
EMF_HIGH_SPD_OFFSET_RISING	BEMF zero-crossing point at rising edge in high-speed mode (unit: voltage digital conversion value)
EMF_HIGH_SPD_OFFSET_FALLING	BEMF zero-crossing point at falling edge in high-speed mode (unit: voltage digital conversion value)

Note [2]: Approximate curve equation of voltage vs. temperature is $V[V] = V_0 + dV/dT[V/Celsius] * (T - T_0)[Celsius]$.

Note [3]: It is applicable to AT32 MCUs with internal CMPs, such as AT32M412.

- Table 7 presents the motor parameters used in this demo, including number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 7. Motor parameter definition

Macro definition	Description
RS_LL	Line-to-line winding resistance (unit: Ω)
LS_LL	Line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	Motor KE value
NOMINAL_CURRENT	Motor rated current (unit: ampere)
ANGLE_INIT_DETECT_DUTY	PWM DUTY value for initial angle detection (unit: PWM counter timing base)
ANGLE_INIT_I_DIFF	Current difference value for initial angle detection (unit: ADC digital value)
HALL_LEARN_DIR	Motor direction after Hall sensor self-learning (note [3])
HALL_LEARN_0_STATE	State 0 after HALL sensor self-learning (BLDC: output V-phase upper-side PWM and W-phase lower-side ON) (note [3])
HALL_LEARN_1_STATE	State 1 after HALL sensor self-learning (BLDC: output V-phase upper-side PWM and U-phase lower-side ON) (note [3])
HALL_LEARN_2_STATE	State 2 after HALL sensor self-learning (BLDC: output W-phase upper-side PWM and U-phase lower-side ON) (note [3])
HALL_LEARN_3_STATE	State 3 after HALL sensor self-learning (BLDC: output W-phase upper-side PWM and V-phase lower-side ON) (note [3])
HALL_LEARN_4_STATE	State 4 after HALL sensor self-learning (BLDC: output U-phase upper-side PWM and V-phase lower-side ON) (note [3])
HALL_LEARN_5_STATE	State 5 after HALL sensor self-learning (BLDC: output U-phase upper-side PWM and W-phase lower-side ON) (note [3])

Note [3]: There is no need to change state in sensorless control mode.

2) `mc_hwio.h` file

- This file is used to configure hardware IOs and peripherals. It also contains the declaration of the `mc_hwio.c` files. Table 8 and Table 9 present the configuration of peripherals (ADC, TMR, USART and EXINT, etc.), interrupt functions and DMA channels that are used in this demo.

Table 8. Configuration of peripherals (AT32F413RCT7)

Peripheral	Interrupt function	DMA channel	Description
ADC1	N/A	DMA1_CH1	ADC1 ordinary channel conversion (BUS voltage, MOS temperature, external speed/torque command)
ADC1	ADC1_2_IRQn	N/A	ADC1 preemptive channel conversion (bus current, BEMF)
TMR1	TMR1_OVF_TMR10_IRQn	N/A	BLDC control interrupt
TMR1	TMR1_CH_IRQHandler	N/A	Disable BEMF pull-up resistor GPIO interrupt (ARTERY patent)
TMR1_CH4	N/A	DMA1_CH4	Current sampling and BEMF sampling time point trigger signals (trigger ADC1 preempted channels)
TMR1	TMR1_BRK_TMR9_IRQn	N/A	PWM output disable
TMR4	TMR4_GLOBAL_IRQn	N/A	Trigger phase change interrupt
USART1_TX	N/A	DMA1_CH2	USART1 TX data
USART1_RX	N/A	DMA1_CH3	USART1 RX data
EXINT	EXINT15_10_IRQn	N/A	START/STOP button (start/stop motor)

Table 9. Configuration of peripherals (AT32F421C8T7)

Peripheral	Interrupt function	DMA channel	Description
ADC1	N/A	DMA1_CH1	ADC1 ordinary channel conversion (BUS voltage, MOS temperature, external speed/torque command)
ADC1	ADC1_CMP_IRQn	N/A	ADC1 preemptive channel conversion (bus current)
TMR1	TMR1_BRK_OVF_TRG_HALL_IRQn	N/A	BLDC control interrupt, PWM output disable
TMR1	TMR1_CH_IRQHandler	N/A	Disable BEMF pull-up resistor GPIO interrupt (ARTERY patent)
TMR1_CH4	N/A	DMA1_CH4	Current sampling and BEMF sampling time point trigger signals (trigger ADC1 preempted channels)
TMR15	TMR15_GLOBAL_IRQn	N/A	Trigger phase change interrupt
USART1_TX	N/A	DMA1_CH2	USART1 TX data
USART1_RX	USART1_IRQn	DMA1_CH3	USART1 RX data
EXINT	EXINT15_4_IRQn	N/A	START/STOP button (start/stop motor)

3) mc_hwio.c file

- This file configuration depends on users' hardware peripherals, such as TMR, ADC, DMA and GPIO. It also contains functions such as button, LED, PWM ON/OFF. See Table 10 for details.

Table 10. Peripheral configuration functions

Function	Description
nvic_config	Interrupt priority configuration
tmr_pwm_init	PWM timer, crm clock, GPIO and DMA configuration
gpio_hall_init	Hall sensor GPIO configuration
adc_ordinary_config	ADC ordinary channel ADC, DMA and GPIO configuration
adc_preempt_config	ADC preemptive channel ADC, DMA and GPIO configuration
uart_init	UART crm clock, GPIO and UART configuration
button_exint_init	Button interrupt event EXINT configuration
button_switch_init	Dip switch configuration
led_config	LED GPIO initialization and crm clock configuration
led_on	LED ON
led_off	LED OFF
led_toggle	LED toggles (from ON to OFF, or from OFF to ON)
led_blink	LED blinks
led_init	LED initialization configuration
current_offset_tmr_setting	Timer configuration for current offset calibration
bldc_angle_init_config	TMR configuration for initial angle detection
tmr_sensorless_change_phase_init	TMR configuration for sensorless commutation
bldc_sensorless_detectEMF_config	TMR, ADC, and DMA configuration for sensorless zero-crossing detection
get_int_vref_cal_ratio	Detect VDDA reference voltage ratio, used for ADC value calibration
motor_parameter_ID_config	Set TMR and ADC relating to motor winding parameter identification
pull_up_config	ARTERY's patent pull-up circuit GPIO configuration (for zero crossing point detection by ADC)
tmr_read_emf_init	TMR configuration for sensorless zero-crossing detection in continuous sampling mode
hwdiv_config	Hardware divider configuration (for AT32L021 only)
opa_init	Bus current OP configuration (for AT32 MCUs with internal OPs, such as AT32M412)
opa_calibration	Bus current OP calibration (for AT32 MCUs with internal OPs, such as AT32M412)
ocp_cmp_config	Bus current protection CMP configuration (for AT32 MCUs with internal CMPs, such as AT32M412)

cmp2_config	BEMF internal CMP configuration (for AT32 MCUs with internal CMPs, such as AT32M412)
tmr_comp_capture_init	BEMF internal CMP TMR capture channel configuration (for AT32 MCUs with internal CMPs, such as AT32M412)
tmr_blank_trigger_init	Internal CMP blanking trigger source TMR configuration (for AT32 MCUs with internal CMPs, such as AT32M412)
tmr_blank_init	Internal CMP blanking signal TMR configuration (for AT32 MCUs with internal CMPs, such as AT32M412)

4) mc_isr.c file

- This file contains interrupt handler functions for ADC, TMR and SYSTICK interrupts, etc. Refer to Table 11.

Table 11. Motor control interrupt functions

Function	Description
ADVTMR_PWM_CYCLE_IRQ	TIMER1 update interrupt function: voltage loop control, current loop control, current & BEMF sampling time counting, UI data transfer
ADVTMR_PWM_BRK_IRQ	TMR1 brake input interrupt function: PWM output disable
ADC_SHUNT_SAMP_READY_IRQ	ADC1 interrupt function: current sampling, BEMF sampling, speed calculation
CHANGE_PHASE_IRQ	Sensorless commutation interrupt function: trigger PWM phase change by software
SysTick_Handler	System interrupt function (1ms): state machine, speed control
BUTTON_EXINT_IRQHandler	External IO interrupt function: START/STOP button (start/stop motor)
ADVTMR_CH1_EMF_PULL_IRQ	Interrupt function for zero-crossing point detection with ADC in sensorless mode (ARTERY's patent pull-up circuit)
READ_EMF_IRQ	Sensorless zero-crossing detection interrupt in continuous sampling mode (for BLDC continuous sampling mode)

5) mc_type.h file

- This file contains global emulation/type definitions. Refer to Table 12 for details.

Table 12. Motor control library emulation list

Item	Description
long_words_union	Singed number 16-bit and 32-bit integer variable union declaration
float_u32_union	Floating-point number and unsigned number 32-bit integer variable union declaration
firmware_id_type	Firmware ID type used to identify motor control mode
motor_control_mode	Motor control mode (open loop control, voltage control, Id control mode, Iq control mode, speed control, torque control, position control, encoder alignment mode, etc.)
ctrl_source_type	Control source (software control, external circuit control)
process_state_type	Process state
esc_state_type	Motor control process state machine
start_state_type	State machine for initial angle detection (for sensorless BLDC only)
err_code_type	Motor error type (over-voltage, under-voltage, over-temperature, overcurrent, encoder error, Hall error, winding parameter identification failure, hall self-learning failure)
shunt_nbr_type	Shunt current sensing mode (1-shunt, 2-shunt or 3-shunt)
low_spd_ctrl_type	Low-speed voltage control related variables
calibrate_process_type	PWM input control neutral calibration process type
switch_mode_type	Forward-reverse mode /forward-brake mode /forward-reverse-brake mode
init_current_type	Current type for initial angle detection
angle_init_type	Initial angle detection type
curr_offs_type	ADC current sampling offset value
i_bus_type	BUS current type
current_type	Current variables
pid_ctrl_type	PID control loop variables
olc_type	Open loop type
speed_type	Speed variables
value_type	Value type
hall_sensor_type	Hall sensor variables
emf_sample_type	BEMF sampling related structure variable type (for sensorless BLDC only)
adc_sample_type	ADC sampling related structure variable type
ramp_cmd_type	Ramp command type
usart_data_index	UART queue variables
moving_average_type	Moving average related structure variable
lowpass_filter_type	Low-pass filter related variables

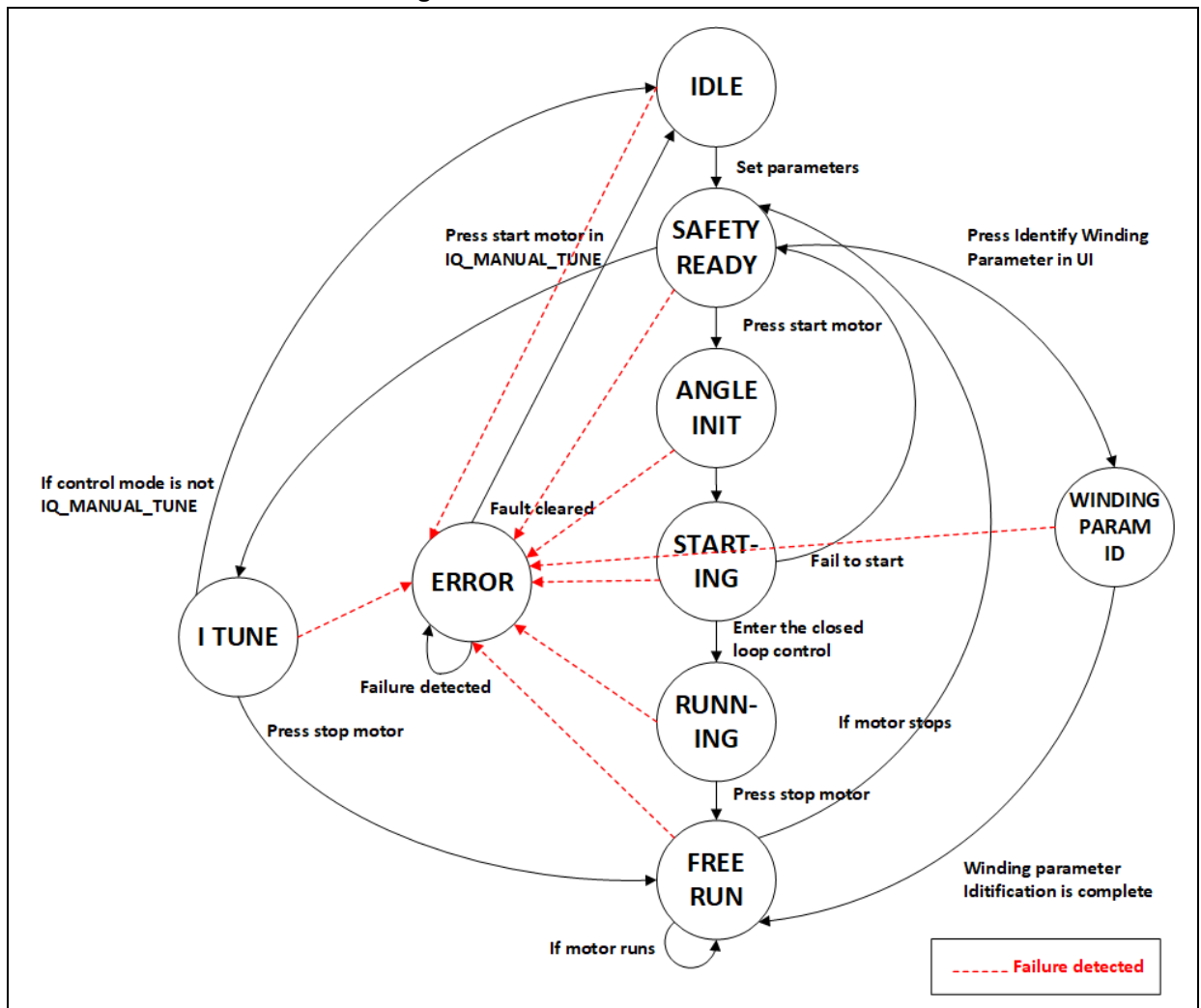
usart_config_type	UART peripheral configuration variable
ui_wave_param_type	UI waveform display variable
i_auto_tune_type	Tune current PI controller parameters
motor_param_id_type	Winding parameters identification
pwm_in_type	PWM input capture related variable
sound_type	Warning tone related variable
blank_type	Internal CMP blanking related variable
blank_trigger_type	Internal CMP blanking trigger source related variable

4 Motor control state machine

4.1 Introduction

Figure 9 illustrates the flow chart of the state machine. The initial settings of each state are checked by the Main program on a circular basis. Upon state change, the new state is programmed with initial settings. In addition, the SysTick Handler interrupt function is responsible for real-time monitoring the state machine at a millisecond rate. Once failure is detected, the state machine enters “Error” state to halt motor so as to avoid potential damage to motor or driver. The motor is being stalled until the failure is cleared.

Figure 9. State machine flow chart



The entire flow chart is composed of Idle, Safety ready, Running, Free run, I_tune and Error states.

Idle

The initial state of a state machine. Motor stays idle in this state. And the state machine returns to "Idle" state when a failure is cleared or motor stops running.

Safety ready

This state indicates that the motor can start up safely as all parameters are already set and the current offset value is available.

Angle init

This refers to the state for initial angle detection prior to motor startup in sensorless control mode. The procedure jumps to "Starting" state as long as the initial angle is obtained. Note that "Starting state" is applicable to the sensorless control mode only. If there is no need to detect initial angle, the "Starting state" can be skipped.

Starting

In sensorless control mode, after the initial angle is detected, the procedure jumps to "Starting state" in which the user can set driver parameters, peripherals and conditions to switch back to closed loop.

Running

This state indicates that the motor is running, and the user can adjust parameters (target speed, target current) or stop motor via UI interface.

Free run

This state is similar to Stop mode. In this state, the driver stops output, and the motor speed will slowly reduce to zero. This state is maintained until the motor stops running. After a full stop of motor, the state machine then returns to "Safety ready" state.

I_tune

This state is used to tune PID controller parameters. In this state, the user can tune the target current, current loop KP and KI values via UI software to generate a step current.

Winding param Id

This refer to the state of motor winding parameter identification. The user can enable this feature via UI interface. The winding parameter identification feature is often used for tuning a new motor. The tuned parameters can be used for sensorless vector control and current PI controller parameters self-tuning.

Auto learn

This refers to a hall state sequence self-learning feature in our motor control library. It can be used in six-step square wave mode and vector control mode.

Error

The state machine jumps to "Error" state in case of any failures.

Table 13 presents the initial state, end state and switch conditions between two states.

Table 13. State switch conditions

Initial state	End state	State switch condition
IDLE	SAFETY REDAY	The control source is from UI tool, and initial parameters are already set.
		The control source is from the external circuit. The POTENTIOMETER value is less than the default value, and initial parameters are already set.
SAFETY REDAY	ANGLE_INIT	When the control mode is SPEED_CTRL: If the control source is from UI tool, the speed command is not 0 and click the START MOTOR (UI tool). If the control source is from the external circuit, the POTENTIOMETER value is greater than the set value and press the START/STOP button on the motor control board.
		When the control mode is TORQUE_CTRL: If the control source is from UI tool, the speed command is not 0 and click the START MOTOR (UI tool). If the control source is from the external circuit, the POTENTIOMETER value is greater than the set value and press the START/STOP button on the motor control board.
	STARTING	When the control mode is OPEN_LOOP_CTRL: If the control source is from UI tool, click the START MOTOR (UI tool). If the control source is from the external circuit, press the START/STOP button on the motor control board.
	I TUNE	When the control mode is IQ_MANUAL_TUNE, click the START MOTOR (UI tool), or press the START/STOP button on the motor control board.
	WINDING_PARAM_ID	Click the "Winding parameter identification" on UI tool
ANGLE_INIT	STARTING	Initial rotor position estimation is completed or rotor alignment is finished
	FREE_RUN	If the control source is from UI tool, click the STOP MOTOR (UI tool). If the control source is from the external circuit, press the START/STOP button on the motor control board.
STARTING	RUNNING	When the control mode is OPEN_LOOP_CTRL
		When the control mode is not OPEN_LOOP_CTRL, the phase change time exceeds a certain value
	SAFETY REDAY	No phase change is detected during the specified period
	FREE_RUN	If the control source is from UI tool, click the STOP MOTOR (UI tool). If the control source is from the external circuit, press the

		START/STOP button on the motor control board.
RUNNING	FREE_RUN	If the control source is from UI tool, click the STOP MOTOR (UI tool). If the control source is from the external circuit, press the START/STOP button on the motor control board.
FREE_RUN	SAFETY REDAY	When the actual speed is 0
ERROR	IDLE	Failure is cleared.
I_TUNE	FREE_RUN	Click the STOP MOTOR (UI tool) or press the START/STOP button on the motor control board.
AUTO_LEARN	FREE_RUN	Hall state sequence self-learning is completed.
WINDING_PARAM_ID	FREE_RUN	Auto identification of motor winding parameters is completed.

5 How to use demo project

For details, refer to *AN0214_AT32_BLDC_Sensorless_Quick_Start_Guide*.

6 Revision history

Table 14. Document revision history

Date	Version	Revision note
2023.03.02	2.0.0	Initial release.
2023.04.27	2.0.1	1. Added open loop startup function 2. Upgraded UI interface (baud rate:1.5M) 3. Updated definitions 4. Added BEMF compensate function
2023.10.05	2.1.0	Modified document structure, updated parameters and UI interface.
2024.05.17	2.1.1	Added parameter auto identification and updated parameters.
2024.12.27	2.1.2	Added applicable products, updated state machine and some parameters, and updated Flash ROM configuration.
2025.10.09	2.1.3	Modified Flash ROM configuration in Table 2, updated some parameters, modified the state machine description, and merged the CMP and ADC project files.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.

© 2025 ARTERY Technology – All Rights Reserved